

2.1

定义1: 若 $\exists c, n_0 > 0$, 使得 $N \geq n_0$ 时 $T(N) \leq cf(N)$, 则 $T(N) = O(f(N))$ 大O表示上界

定义2: 若 $\exists c, n_0 > 0$, 使得 $N \geq n_0$ 时 $T(N) \geq cf(N)$, 则 $T(N) = \Omega(f(N))$ Ω 表示下界

定义3: 当且仅当 $T(N) = O(f(N))$ 且 $T(N) = \Omega(f(N))$ 时, $T(N) = \Theta(f(N))$ Θ 表示相当

eg: 设 $f(N) = 3N$, 则 $f(N) \leq 5N$, $f(N) = O(N)$, 且 $f(N) \geq N$, $f(N) = \Omega(N)$ 则称 $f(N) = \Theta(N)$

定义4: 若 $T(N) = O(f(N))$ 且 $T(N) \neq \Theta(f(N))$, 则称 $T(N) = o(f(N))$ 小o表示严格小

法则1: 若 $T_1(N) = O(f(N))$ 且 $T_2(N) = O(g(N))$ 则有

$$T_1(N) + T_2(N) = O(f(N) + g(N)) \quad T_1(N) T_2(N) = O(f(N)g(N))$$

法则2: 若 $T(N)$ 是 k 次多项式, 则 $T(N) = O(N^k)$

法则3: $\forall k \in \mathbb{R}, \log^k N = O(N)$

2.4 计算运行时间

法则1: for 循环运行时间至多是循环体内运行时间 $\times$ 迭代次数

法则2: for 循环嵌套时间为每层循环之积

法则3: 顺序语句求和即可

法则4: if/else 运行时间不超过判断 + 最耗时的分支

例: 求解最大子序列

算法1: 双层嵌套for 循环, 时间复杂度为 $O(N^2)$

算法2: 分治算法, 先把序列分为两半

则最大序列可能是在左半、右半、或横跨两半。

设该算法的时间复杂度为 $T(N)$, 则左半和右半均为 $T(\frac{N}{2})$

横跨两半需要分别计算从中间到两边的最大子序列。

通过两个for 循环实现 时间复杂度为两个 $O(N)$ 相加

则有 $T(N) = 2T(\frac{N}{2}) + O(N)$, 方程解为 $T(N) = O(N \log N)$

算法3: 先逐个累加, 若和为负, 则对于后面肯定不利于形成最大子序列

故将前面置为0, 后面继续, 时间复杂度为 $O(N)$

二分查找：直观地理解，若有 2^k 个数，至多查找 $k+1$ 次，时间复杂度是 $O(\log N)$

欧几里德算法（辗转相除法）：

经过两次迭代后，余数至多是原始值的一半，即规模减半，时间复杂度是 $O(\log N)$

下证：若 $M > N$ ，则 $M \% N < \frac{M}{2}$

① 若 $N \leq \frac{M}{2}$ ，则 $M \% N < N \leq \frac{M}{2}$ 成立

② 若 $N > \frac{M}{2}$ ，则 $M/N = 1$ ，故 $M \% N = M - N < \frac{M}{2}$ 成立

高效幂运算（节约乘法次数）

若 N 为偶数，则 $x^N = x^{\frac{N}{2}} \cdot x^{\frac{N}{2}}$ ，若 N 为奇数，则 $x^N = x^{\frac{N-1}{2}} \cdot x^{\frac{N-1}{2}} \cdot x$

显然，至多需进行两次乘法即可将规模减半，时间复杂度为 $O(\log N)$

二进制快速求幂。

大致思路：定义 base 每次迭代为 x ， x^2 ， x^4 ， $x^8 \dots$

每次将 n 折半，相当于二进制右移。这时根据 $n \% 2$ （二进制最右一位）

判断是否乘 base，直到 $n=0$ 时停止。

练习

2.1

$$\frac{2}{3} < 3 < \sqrt{N} < N < N \log \log N < N \log N = N \log N^2 < N \log^2 N < N^{1.5} < N^2 < N^2 \log N < N^3 < 2^{\sqrt{N}} < 2^N$$

2.2 a ✓ b ✗ c ✗ d ✗ 太0不一定是最近的

$$2.3 \quad N \log N \sim N^{\epsilon} \left(1 + \frac{\epsilon}{\sqrt{\log N}}\right)$$

$$\log N \sim N^{\epsilon} \left(\frac{\epsilon}{\sqrt{\log N}}\right)$$

$$\log \log N \sim \epsilon \sqrt{\log N}$$

$\log t \sim \epsilon \sqrt{t}$ 故 RHS 增长更快.

$$2.5 \quad f(N) = \begin{cases} N, & N \text{ 奇} \\ N^2, & N \text{ 偶} \end{cases} \quad g(N) = \begin{cases} N^2, & N \text{ 奇} \\ N, & N \text{ 偶} \end{cases}$$

$$2.6 \quad a. f(N) = f(N-1)^2 \quad b. D = 2^{2^{N-1}}$$

$$\log f(N) = 2 \log f(N-1)$$

$$N = \log \log D + 1 = O(\log \log D)$$

$$\therefore \log f(N) = 2^{N-1} \cdot \log f(1) = 2^{N-1}$$

$$\Rightarrow f(N) = 2^{2^{N-1}}$$

$$2.7 \quad (1) O(N) \quad (2) O(N^2) \quad (3) O(N^3) \quad (4) O(N^2) \quad (5) O(N^5) \quad (6) O(N^4)$$

2.8

$$\text{算法1: } T(N) = \sum_{i=0}^{N-1} \frac{N}{N-i} = O(N^2 \log N)$$

$$\text{算法2: } T(N) = \sum_{i=0}^{N-1} \frac{N}{N-i} = O(N \log N)$$

$$\text{算法3: } T(N) = O(N)$$

2.14 证明合理性

$$\begin{aligned} f(x) &= a_0 + a_1 x + a_2 x^2 + \dots + a_n x^n \\ &= a_0 + x(a_1 + a_2 x + \dots + a_n x^{n-1}) \\ &= a_0 + x(a_1 + x(a_2 + \dots + a_n x^{n-2})) \end{aligned}$$

2.21

真正花时间的地方是删除 (置为 false)

对于一个质数 p , 需要删除 $2p, 3p, \dots$ 共 $\frac{N}{p}$ 个数

考虑到 $p > \sqrt{N}$ 时, 后面的合数已被删除, 无需重复.

$$\therefore T(N) = \sum_{p \leq \sqrt{N}} \frac{N}{p} = N \cdot \ln \ln \sqrt{N} = N \cdot \ln \left(\frac{\ln N}{2} \right) = O(N \log \log N)$$

2.26

教材解法：分治法

A 3 3 4 2 4 4 2 4 4
 B 3 4

两两组合，若两数相等则放入B，则B中元素个数 $\leq A$ 的一半

每层扫描时间依次为 $N, \frac{N}{2}, \frac{N}{4} \dots$ 故时间复杂度为 $O(N)$

如果是奇数，则先把落单的拿出来，看前面偶数个返回的 candidate

如果选出 candidate，就直接去验证

如果 candidate 未选出（返回 -1），就返回落单元素

如果落单元素就是主元素，那验证环节会呈现，否则没有主元素

关于 candidate 能否选出，看最后一个数组

如果只有一个元素，则占比超过一半，认为是候选元。

如果没有元素，则上一个容器里为两个不等元素，都不认为是主元素

下证 B 中主元素也为 A 中主元素

设主元素为 M，考虑下面 4 种配对类型

类型	数目	
$M \sim M$	p	放 B
$M \sim x$	q	
$x \sim x$	r	放 B
$x \sim y$	s	

$$\text{则 } 2p + q > \frac{1}{2}(2p + 2q + 2r + 2s) \Rightarrow p > r + s \Rightarrow p > r$$

故放入 B 后 M 有 p 个，非 M（还可能不同）有 r 个，M 仍为主元素。

显然，这种算法实在复杂，更好的算法是众数投票法